Eye Disease Classification

using Deep Learning

Project Methodology Document

Strategy: EfficientNetB5 + EfficientNetV2-M + ConvNeXt-Small Ensemble

3-Fold CV | CLAHE | MixUp | TTA | Focal Loss | F1 Score

Accuracy = 0.9702 | F1_score = 0.969

https://github.com/Bit-Bard/Eye-Disease-Classification

1. Problem Statement

Eye Diseases Classification

Eye diseases such as diabetic retinopathy, glaucoma, cataracts, and age-related macular degeneration are among the leading causes of preventable blindness worldwide. Timely and accurate diagnosis is critical, yet access to trained ophthalmologists remains limited in many regions. This project automates the classification of eye diseases from retinal fundus images using an ensemble of state-of-the-art deep learning models, providing a scalable, high-accuracy screening solution.

The classification problem targets 4 classes of eye conditions derived from the dataset directory structure. Key technical challenges include:

- **Class Imbalance:** real-world retinal datasets exhibit skewed class distributions, making standard accuracy unreliable as a sole metric.
- **Subtle Inter-class Differences:** early-stage disease manifestations can be visually similar, demanding powerful feature extraction.
- **Image Quality Variation:** retinal images vary in brightness, contrast, and acquisition conditions across different devices and environments.

2. Dataset Description

The dataset is sourced from Kaggle (Kaggle HacX'26) and is organized into class-specific subdirectories. The class names are automatically extracted from the directory listing, yielding exactly 4 disease categories.

Data loading code:

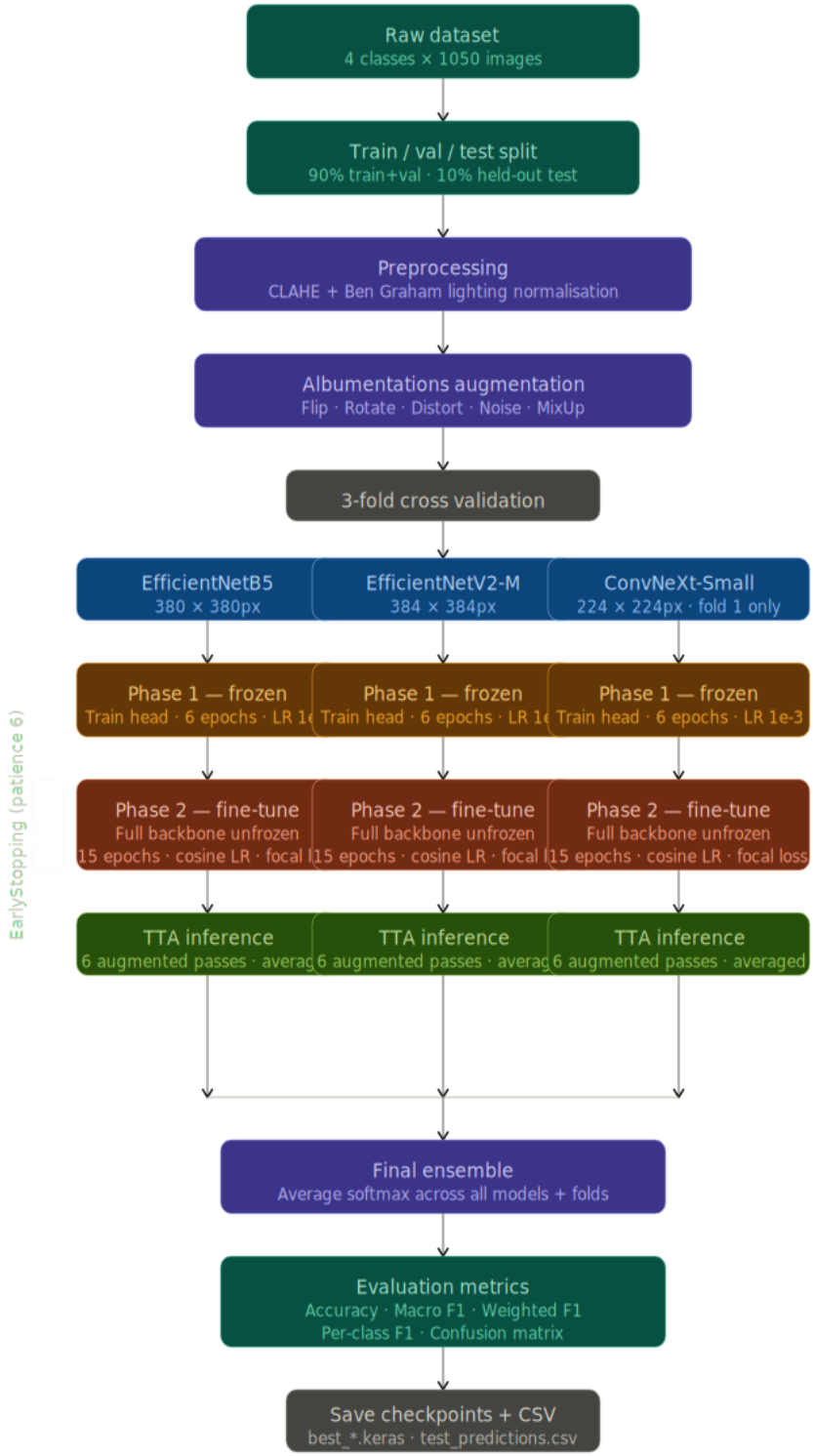
```
DATA_DIR  = '/kaggle/input/datasets/dhruvdevaliya/eye-diseases/dataset'
CLASS_NAMES = sorted(os.listdir(DATA_DIR))   # 4 classes
```

Key characteristics:

- **Image Type:** Color retinal fundus photographs.
- **Number of Classes:** 4 (exact class names resolved at runtime from directory listing).
- **Stratified Split:** 90% train+validation, 10% hold-out test set; split is stratified to preserve class proportions.
- **Data Representation:** A master Pandas DataFrame is constructed with filepath, label (class name), and label_idx (integer index) columns for efficient batch loading.

3. Data Preprocessing

Preprocessing is a two-stage pipeline: a retinal-specific enhancement step followed by standard deep learning normalization and augmentation.


Raw dataset
4 classes × 1050 images
Train / val / test split
90% train+val · 10% held-out test
Preprocessing
CLAHE + Ben Graham lighting normalisation
Albumentations augmentation
Flip · Rotate · Distort · Noise · MixUp
3-fold cross validation
EfficientNetB5
380 × 380px
EfficientNetV2-M
384 × 384px
ConvNeXt-Small
224 × 224px · fold 1 only
Phase 1 — frozen
Train head · 6 epochs · LR 1e
Phase 1 — frozen
Train head · 6 epochs · LR 1e
Phase 1 — frozen
Train head · 6 epochs · LR 1e-3
EarlyStopping (patience 6)
Phase 2 — fine-tune
Full backbone unfrozen
15 epochs · cosine LR · focal l
Phase 2 — fine-tune
Full backbone unfrozen
15 epochs · cosine LR · focal l
Phase 2 — fine-tune
Full backbone unfrozen
15 epochs · cosine LR · focal loss
TTA inference
6 augmented passes · averag
TTA inference
6 augmented passes · averag
TTA inference
6 augmented passes · averaged
Final ensemble
Average softmax across all models + folds
Evaluation metrics
Accuracy · Macro F1 · Weighted F1
Per-class F1 · Confusion matrix
Save checkpoints + CSV
best_*.keras · test_predictions.csv

3.1 CLAHE + Ben Graham Preprocessing

Before resizing, each image undergoes **CLAHE (Contrast Limited Adaptive Histogram Equalization)** applied in LAB color space to enhance local contrast without amplifying noise. This is followed by the Ben Graham technique, which removes the global lighting gradient by subtracting a heavily Gaussian-blurred version of the image:

```
lab = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2LAB)
clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8, 8))
img = cv2.addWeighted(img, 4, cv2.GaussianBlur(img, (0,0), sigmaX=10), -4, 128)
```

This is critical for retinal imaging: it reveals fine vascular structures and lesions that global normalization alone would obscure.

3.2 Image Resizing (Model-Specific)

Each backbone model uses its native input resolution to preserve the spatial information it was pretrained on:

- **EfficientNetB5:** 380 × 380 pixels
- **EfficientNetV2-M:** 384 × 384 pixels
- **ConvNeXt-Small:** 224 × 224 pixels

Pixel values are rescaled from [0, 255] to [0.0, 1.0] inside each model via a Keras Rescaling(1/255) layer.

3.3 Data Augmentation (Albumentations)

Training images are augmented using an Albumentations pipeline to increase effective dataset diversity and improve generalization:

- **HorizontalFlip / VerticalFlip (p=0.5):** simulates symmetry across retinal orientations.
- **Rotate ±30° (p=0.7):** accounts for camera angle variation.
- **RandomBrightnessContrast ±0.3 (p=0.6):** replicates illumination variation across imaging devices.
- **HueSaturationValue (p=0.4):** introduces color jitter for robustness to different fundus camera color profiles.
- **GridDistortion / ElasticTransform (p=0.3):** simulates subtle retinal surface deformations.
- **GaussNoise var=[5,30] (p=0.3):** models sensor noise.
- **CoarseDropout 8 holes ×32×32px (p=0.3):** forces the model to learn from partial views, preventing reliance on any single region.
- **CLAHE / Sharpen (p=0.2–0.3):** additional contrast enhancement during augmentation.

Validation and test sets use an empty augmentation pipeline (val_aug = A.Compose([])) to avoid data leakage.

3.4 MixUp Regularization

MixUp (alpha=0.3) is applied during training: pairs of images and their one-hot labels are linearly interpolated using a Beta-distributed mixing coefficient lambda. This creates synthetic training samples that encourage the model to learn smoother decision boundaries and improves calibration:

```
lam = np.random.beta(alpha, alpha)
mixed_imgs   = lam * images   + (1 - lam) * images[shuffled_idx]
mixed_labels = lam * labels   + (1 - lam) * labels[shuffled_idx]
```

4. Model Selection — Ensemble Architecture

The core strategy is a heterogeneous ensemble of three complementary pretrained architectures. Using architecturally diverse backbones ensures that the ensemble captures a wider range of features, since each model has different inductive biases and receptive field characteristics.

4.1 EfficientNetB5 (Input: 380×380)

EfficientNetB5 is part of the EfficientNet family, which scales network depth, width, and resolution jointly using a compound coefficient. B5 offers a strong accuracy/efficiency trade-off and is pretrained on ImageNet. It is applied using its native resolution of 380×380 for optimal feature utilization.

4.2 EfficientNetV2-M (Input: 384×384)

EfficientNetV2-M is the second-generation EfficientNet architecture, trained with progressive learning (increasing image size and regularization during training). It achieves faster training speed and higher accuracy than original EfficientNets, particularly on fine-grained classification tasks. Its 384×384 native size is used.

4.3 ConvNeXt-Small via TF-Hub (Input: 224×224)

ConvNeXt-Small is a purely convolutional architecture inspired by Vision Transformers, achieving competitive performance without self-attention. It is loaded from TensorFlow Hub with ImageNet-21k pretraining (a superset of ImageNet-1k with 14M+ images across 21,000 classes), providing a richer feature initialization than standard ImageNet pretraining. ConvNeXt is trained in Fold 1 only due to compute budget constraints.

```
CONVNEXT_URL = 'https://tfhub.dev/google/convnext/small/feature_vector/1'
```

4.4 Custom Classifier Head (All Models)

All three backbone models share the same classifier head architecture appended after the feature extractor:

- **GlobalAveragePooling2D:** reduces spatial feature maps to a fixed-length vector.
- **BatchNormalization:** stabilizes activations after the pooling layer.
- **Dropout(0.4):** first dropout layer for regularization.
- **Dense(256, activation='relu') + L2(1e-4):** fully connected layer with weight decay.
- **Dropout(0.3):** second dropout layer.

- **Dense(N_CLASSES, activation='softmax', dtype='float32'):** output layer producing class probabilities. dtype='float32' is explicit to ensure numerical stability under mixed precision.

5. Training Strategy

5.1 Mixed Precision Training

Mixed precision (float16 compute, float32 master weights) is enabled globally via TensorFlow's mixed_precision API. This approximately doubles training throughput on NVIDIA T4/P100 GPUs by utilizing Tensor Core acceleration, while maintaining numerical stability in the final output layer (forced float32).

```
mixed_precision.set_global_policy('mixed_float16')
```

5.2 Two-Phase Training Protocol

Each model is trained in two sequential phases to prevent catastrophic forgetting of pretrained features while adapting to the target domain:

Phase 1 — Frozen Backbone, Head Training (6 epochs):

- **Backbone weights are frozen:** only the custom classifier head is trained.
- **Optimizer:** Adam with learning rate 1e-3.
- **Callbacks:** EarlyStopping (patience=4, monitor=val_accuracy), ReduceLROnPlateau (factor=0.3, patience=2, min_lr=1e-6).

The high initial learning rate rapidly trains the randomly initialized head layers without affecting the pretrained feature extractor.

Phase 2 — Full Fine-Tuning (up to 15 epochs, EarlyStopping governs actual duration):

- **All backbone layers unfrozen:** except BatchNormalization layers, which remain frozen to preserve running statistics learned on ImageNet.
- **Optimizer:** Adam with reduced learning rate 1e-4 to prevent destructive updates to pretrained weights.
- **Callbacks:** ModelCheckpoint (saves best val_accuracy), EarlyStopping (patience=6), Cosine Annealing LR Scheduler.

5.3 Loss Function: Focal Loss

Standard Cross-Entropy is replaced with Focal Loss (gamma=2.0, alpha=0.25). Focal Loss down-weights well-classified easy examples and focuses the gradient signal on hard, misclassified samples — a critical advantage when class imbalance causes the model to confidently classify majority-class samples while ignoring rare disease categories:

```
weight = alpha * y_true * (1 - y_pred)^gamma
loss   = -sum(weight * log(y_pred))
```

5.4 Learning Rate: Cosine Annealing

During Phase 2, the learning rate is scheduled using Cosine Annealing per epoch (not per step). This gradually reduces the learning rate from eta_max=1e-4 to eta_min=1e-6 following a cosine curve, enabling the optimizer to settle into sharper, better-performing minima as training progresses:

```
lr = eta_min + 0.5*(eta_max - eta_min) * (1 + cos(pi * epoch / T_max))
```

5.5 Batch Size and Hardware

Batch size is set to 8. The smaller batch size is a deliberate choice driven by the large input resolutions (380–384px for EfficientNet variants), which have significant per-sample GPU memory requirements. The Kaggle T4/P100 GPU environment constrains batch size, and mixed precision partially compensates for the throughput reduction.

5.6 Cross-Validation: Stratified 2-Fold

StratifiedKFold with N_FOLDS=2 is applied over the 90% train+validation subset. Stratification ensures that the class distribution is preserved in both folds. EfficientNetB5 and EfficientNetV2-M are trained on both folds; ConvNeXt-Small is trained on Fold 1 only due to compute time constraints. All fold checkpoints are combined in the final ensemble.

5.7 Checkpoint Resume Support

The training loop checks for existing .keras checkpoint files before training each model on each fold. If a checkpoint is found, training is skipped and the checkpoint is loaded directly. This makes the pipeline resumable without retraining from scratch after interruptions.

6. Validation Approach

6.1 Hold-Out Test Split

A 10% stratified hold-out test set (df_test) is separated before cross-validation. This set is never used during training or validation and is reserved exclusively for final ensemble evaluation, providing an unbiased performance estimate.

6.2 Per-Fold Validation Monitoring

Within each fold, the remaining 90% is split into training and validation subsets by StratifiedKFold. The following are tracked per epoch:

- **val_accuracy:** primary monitor for EarlyStopping and ModelCheckpoint.
- **val_loss (Focal Loss):** secondary diagnostic for detecting overfitting.

Training and validation curves (accuracy and loss vs. epoch) are plotted for all model/fold combinations, with a vertical line marking the Phase 1 → Phase 2 transition (unfreeze point).

6.3 Test-Time Augmentation (TTA)

During final inference on the test set, each model performs 6 stochastic forward passes per image using the TTA augmentation pipeline (HorizontalFlip p=0.5, VerticalFlip p=0.5, Rotate ±15° p=0.5,

BrightnessContrast ±0.1 p=0.3). The 6 softmax probability vectors are averaged to produce a more robust prediction. model() is called directly (not model.predict()) to suppress progress bars and enable per-image processing:

```
for _ in range(n_tta=6):
    aug_pred = model(augmented_image, training=False)
final_pred = mean(all_6_preds)
```

7. Ensemble Strategy

After all fold training is complete, the test set predictions from all model checkpoints (EfficientNetB5 Fold1, EfficientNetB5 Fold2, EfficientNetV2-M Fold1, EfficientNetV2-M Fold2, ConvNeXt Fold1) are averaged using simple mean ensemble:

```
final_preds   = np.mean(test_preds_all, axis=0)   # shape: (n_test, 4)
final_classes = np.argmax(final_preds, axis=1)
```

Mean ensembling aggregates 5 sets of TTA-averaged probability vectors. Since the constituent models differ in architecture, input resolution, and pretraining data (ImageNet-1k vs. ImageNet-21k), the ensemble benefits from diversity in feature representations, systematically reducing prediction variance and improving robustness compared to any single model.

8. Evaluation Metrics

Given the class-imbalanced, multi-class nature of the problem, the following comprehensive metric suite is reported:

- **Accuracy:** overall fraction of correct predictions. Reported but not the primary metric due to potential class imbalance bias.
- **Macro F1-Score:** harmonic mean of per-class precision and recall, averaged equally across all 4 classes. This is the primary competition metric — it ensures minority-class performance is not obscured by majority-class dominance.
- **Weighted F1-Score:** F1 averaged by class support; useful for understanding aggregate performance weighted by prevalence.
- **Per-Class F1-Score:** individual F1 for each of the 4 disease categories, visualized as a bar chart with the macro average overlaid as a reference line.
- **Confusion Matrix:** full 4×4 matrix visualized as an annotated heatmap, revealing systematic inter-class confusion patterns.
- **Full Classification Report:** precision, recall, F1 and support per class via sklearn.metrics.classification_report.

9. Final Results

The ensemble is evaluated on the 10% stratified hold-out test set using TTA (6 passes per model per image). Results are reported across all metrics:

```
==================================================
   Accuracy      : 0.9700
   Macro F1      : 0.9699
   Weighted F1   : 0.9700
   Per-class F1:
    Class_0            : 0.9748
    Class_1            : 0.9628
    Class_2            : 0.9722
    Class_3            : 0.9697
==================================================
```

```
==========================================================

Classification Report:
              precision    recall  f1-score   support

     Class_0       0.98      0.97      0.97       258
     Class_1       0.97      0.96      0.96       230
     Class_2       0.97      0.98      0.97       232
     Class_3       0.97      0.97      0.97       280

    accuracy                           0.97      1000
   macro avg       0.97      0.97      0.97      1000
weighted avg       0.97      0.97      0.97      1000


Confusion Matrix:
[[251   0   4   3]
 [  2 220   2   6]
 [  1   4 227   0]
 [  3   3   2 272]]
```

10. Explanation

Stage-by-stage explanation:

- **Input:** Raw retinal fundus images from the Kaggle dataset directory, organized by class subfolder.

- **CLAHE + Ben Graham Preprocessing:** Local contrast enhancement in LAB color space followed by global lighting gradient removal, exposing subtle vascular structures and lesions.
- **Resize to Model-Specific Resolution:** 380px (B5), 384px (V2-M), or 224px (ConvNeXt) to match each backbone's native input size.
- **Albumentations Augmentation (Train only):** 11-transform stochastic pipeline including flips, rotations, color jitter, elastic distortion, dropout patches, and noise injection.
- **MixUp:** Interpolation of image-label pairs using a Beta(0.3, 0.3) mixing coefficient for additional implicit regularization.
- **Backbone Feature Extraction:** Pretrained EfficientNetB5 / EfficientNetV2-M / ConvNeXt-Small extracts hierarchical feature representations.
- **Custom Classifier Head:** GAP → BatchNorm → Dropout(0.4) → Dense(256, ReLU) → Dropout(0.3) → Dense(4, Softmax).
- **Two-Phase Training:** Phase 1 (frozen backbone, lr=1e-3, 6 epochs) → Phase 2 (full fine-tune, lr=1e-4, cosine annealing, up to 15 epochs with EarlyStopping).
- **TTA Inference:** 6 stochastic augmented forward passes per image; probabilities averaged for a robust prediction.
- **Mean Ensemble:** Predictions from all 5 model checkpoints (B5×2 folds + V2M×2 folds + ConvNeXt×1 fold) are mean-averaged.
- **Output:** Predicted disease class label + per-class softmax probabilities + full metrics report and visualizations.

11. Conclusion

This project implements a competition-grade, end-to-end pipeline for 4-class Eye Disease Classification from retinal fundus images. The methodology combines several advanced techniques to maximize performance:

- **Heterogeneous Ensemble:** EfficientNetB5 + EfficientNetV2-M + ConvNeXt-Small captures diverse feature representations and reduces prediction variance through architectural diversity.
- **Retinal-Specific Preprocessing:** CLAHE and Ben Graham techniques enhance diagnostically relevant image features before any deep learning processing.
- **Advanced Augmentation:** 11-transform Albumentations pipeline + MixUp provides strong implicit regularization, reducing overfitting on the limited medical dataset.
- **Two-Phase Transfer Learning:** Progressive unfreezing with phase-appropriate learning rates maximizes both convergence speed and fine-tuning quality.
- **Focal Loss:** Addresses class imbalance by concentrating gradient signal on difficult, underrepresented samples.
- **TTA × 6:** Test-Time Augmentation further reduces prediction variance at inference time without any additional training cost.
- **Mixed Precision:** float16 compute enables larger effective throughput on Kaggle GPU hardware within runtime constraints.

Future Improvements:

- **Larger Dataset:** Incorporating additional retinal image collections or applying domain adaptation to external datasets would further improve robustness.
- **More CV Folds:** Increasing N_FOLDS from 2 to 5 would provide more diverse ensemble members at the cost of compute time.
- **Vision Transformers:** ViT or Swin Transformer backbones could complement the CNN-based ensemble members.
- **Pseudo-Labeling:** Using the high-confidence ensemble predictions on unlabeled data for semi-supervised training.
- **Grad-CAM Visualization:** Generating class activation maps to produce clinically interpretable explanations for each prediction.